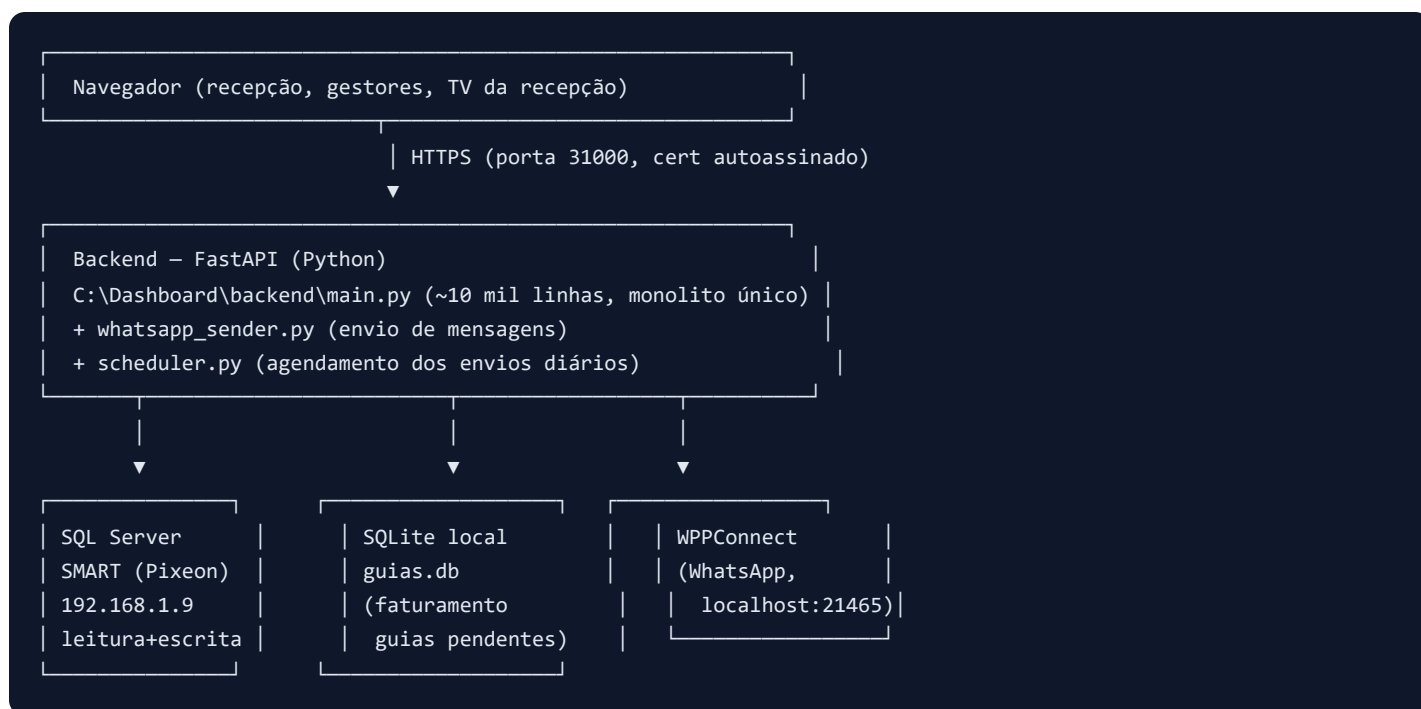


Documentação da Plataforma Dashboard — Visão Geral

O que é

Plataforma de gestão clínica desenvolvida para a ICDS (Instituto de Cooperação para o Desenvolvimento da Saúde), operando sobre o ERP **Smart Píxeon** (banco de dados **SMART**, SQL Server, hospedado em **192.168.1.9**). O Dashboard não substitui o Píxeon — ele lê (e em alguns casos escreve) diretamente nas mesmas tabelas usadas pelo aplicativo desktop da Píxeon, oferecendo relatórios, indicadores de produção/faturamento e algumas funcionalidades novas (fluxo de caixa, lançamento de despesas, monitoramento de pontualidade) que não existem nativamente no Píxeon.

Arquitetura geral



O frontend é uma **Single Page Application** em React, compilada estaticamente (`npm run build` → `frontend/dist`) e servida pelo próprio FastAPI via `StaticFiles` — não existe um servidor Node/Vite rodando em produção; o build tem que ser feito manualmente a cada mudança de frontend (`npm run build` dentro de `C:\\Dashboard\\frontend`) e o resultado é servido direto pelo backend na mesma porta 31000.

Tecnologias utilizadas

Backend

- **Python 3.12+ / FastAPI** — framework HTTP, roda com `uvicorn`.
- **pyodbc** — conexão com o SQL Server do Píxeon (`ODBC Driver 17 for SQL Server`).
- **sqlite3** — banco local (`guias.db`) para o módulo de Faturamento (guias pendentes), independente do SQL Server.

- **httpx** — chamadas HTTP a serviços externos (GLPI, etc).
- **APScheduler-like** (thread próprio, ver `scheduler.py`) — agenda envios diários de WhatsApp.
- **SSL nativo do uvicorn** (`--ssl-keyfile` / `--ssl-certfile`) — HTTPS obrigatório desde 01/07/2026, com certificado autoassinado (`ssl/dashboard_ip.crt`), necessário para permitir acesso externo via túnel Cloudflare sem misturar HTTP/HTTPS.

Frontend

- **React 18 + Vite** — SPA sem client-side router; a navegação entre módulos é feita trocando um `activeModule` em estado local, não por URL.
- **Recharts** — todos os gráficos (linhas, barras, compostos, pizza).
- **Estilo**: majoritariamente objetos de estilo inline (`style={{...}}`), Tailwind está instalado mas pouco usado no Dashboard principal (mais presente no NetMonitor).
- **PWA**: `manifest.json` + `sw.js` (service worker) — o app pode ser "instalado" e tem ícones dedicados.

Infraestrutura

- **Windows Server 2019**, backend e frontend hospedados na mesma máquina.
- **Hyper-V** presente na máquina (virtual switch bindada à placa de rede física) — atenção: isso já causou problemas de conectividade intermitente entre esta máquina e outras na mesma LAN (ver histórico de troubleshooting).
- **Cloudflare Tunnel** (`cloudflared` , modo "quick tunnel") — permite acesso externo sem abrir porta no roteador; a URL gerada é efêmera (muda a cada reinício do túnel).
- **Firewall do Windows**: regra dedicada liberando a porta 31000/TCP (perfil "Any").
- Acesso interno: `https://192.168.1.40:31000` (IP fixo da máquina na LAN) — **sempre HTTPS**, HTTP puro retorna conexão vazia (`ERR_EMPTY_RESPONSE`).

Bancos de dados

- **SMART (SQL Server, Pigeon)** — banco principal, dezenas de tabelas do ERP (agm, fat, mte, smm, osm, CPG/IPG, LOC, PSV, CNV, USR, GR_SES, FLE, EMP, CCT, etc). O Dashboard lê extensivamente e escreve em alguns pontos específicos (ver docs de módulos).
- **guias.db (SQLite)** — só para o controle de guias pendentes de faturamento, independente do Pigeon.

Aplicações irmãs (fora do Dashboard principal, mas relacionadas)

- **NetMonitor** (`C:\NetMonitor`) — monitoramento de infraestrutura (GLPI, páginas de convênios, servidores, link de internet), reaproveita a sessão WhatsApp do Dashboard. Ver `06-netmonitor.md` .
- **Painel de Senhas / Painel TV de Recepção** (`C:\Users\administrator.CENSO\Desktop\painel_recepcao`) — build estático separado, servido pelo Dashboard em `/painel-tv` , com chamada de senha por voz (Web Speech API).
- **wppconnect-server** — servidor local (porta 21465) que mantém a sessão do WhatsApp Business conectada; o Dashboard renova o token automaticamente.

Convenções importantes do código

- Comentários e nomes de variáveis majoritariamente em português.
- Cores padrão por módulo: Produção `#0891B2` (ciano), Ocupacional `#8B5CF6` / `#D97706` , Assistencial `#8B1A1A` (vinho — cor principal da marca), Faturamento `#059669` /vermelho para pendências, sucesso `#10B981` , alerta `#F59E0B` , crítico `#EF4444` .
- Padrão de card branco: `borderRadius:16` , `boxShadow:"0 2px 8px rgba(0,0,0,0.07)"` .
- `periodo_datas(periodo)` é o helper central para converter strings de período ("7d", "30d", "mes:YYYY-MM") em datas de início/fim — reutilizado por quase todos os endpoints financeiros.

- Padrão de segurança para escrita em produção (usado no lançamento de despesas e no registro clínico): investigar schema real antes de codar, testar com valor simbólico, confirmar visualmente, só então liberar de vez.

Índice dos demais documentos

- `02-backend-financeiro-producao.md` — Fluxo de caixa, despesas, valor/hora por médico e consultório, metas.
- `03-backend-clinica-recepcao.md` — Assistencial/Ocupacional/Serviços, agendamentos, recepção, pontualidade.
- `04-backend-faturamento-estoque-whatsapp.md` — Guias, estoque, pacientes, WhatsApp, painel de senhas.
- `05-frontend-arquitetura-modulos.md` — Estrutura do React, módulos e telas.
- `06-netmonitor.md` — Plataforma de monitoramento de infraestrutura.

Gerado automaticamente a partir de 01-visao-geral.md